

DFS AND BFS Visualizer - Project Report

Name: ABHISHEK SARAN

Registration Number: 25MIB10015

Course: CSA-2001 Fundamentals in AI and ML

Semester: 3

Institution: Vellore Institute of Technology,
Bhopal

1. Introduction

1.1 Project Overview

This project is a Python-based visualization tool for graph traversal algorithms, specifically Breadth-First Search (BFS) and Depth-First Search (DFS). The visualizer provides an interactive way to understand how these fundamental algorithms work by showing step-by-step execution with color-coded visual feedback.

1.2 Motivation

Graph traversal algorithms are fundamental concepts in computer science and artificial intelligence. Understanding these algorithms is crucial for:

- Search problems in AI
- Pathfinding algorithms
- Network analysis
- Decision tree traversal
- Social network analysis

Visual representation makes these abstract concepts easier to understand for students.

2. Objectives

The main objectives of this project are:

1. **Educational Tool:** Create an easy-to-understand visualization of BFS and DFS algorithms
2. **Interactive Learning:** Allow users to experiment with different graphs and starting nodes
3. **Visual Feedback:** Use color coding to show algorithm progress in real-time
4. **Simplicity:** Keep the interface simple and beginner-friendly
5. **Flexibility:** Support both pre-built and custom graph structures

3. Technologies Used

3.1 Programming Language

- **Python 3.x** - Chosen for its simplicity and extensive library support

3.2 Libraries

- **NetworkX:** Graph data structure and layout algorithms
- **Matplotlib:** Visualization and plotting
- **Collections (deque):** Queue implementation for BFS

3.3 Why These Technologies?

- Python is beginner-friendly and widely used in AI/ML
- NetworkX provides built-in graph handling
- Matplotlib offers easy visualization capabilities
- All libraries are open-source and well-documented

4. Algorithm Implementation

4.1 Breadth-First Search (BFS)

Concept: BFS explores the graph level by level, visiting all neighbours of a node before moving to the next level.

Data Structure Used: Queue (FIFO - First In First Out) **Steps:**

1. Start from the initial node and mark it as visited
2. Add starting node to the queue
3. While queue is not empty:
 - Remove a node from the front of the queue
 - Visit all unvisited neighbours
 - Add unvisited neighbours to the queue
 - Mark them as visited

Time Complexity: $O(V + E)$ where V = vertices, E = edges

Use Cases:

- Finding shortest path in unweighted graphs
- Level-order traversal
- Social network connections (friend suggestions)

4.2 Depth-First Search (DFS)

Concept: DFS explores as far as possible along each branch before backtracking.

Data Structure Used: Stack (implemented via recursion) **Steps:**

1. Start from the initial node and mark it as visited
2. Recursively visit an unvisited neighbour
3. Backtrack when no unvisited neighbours are available
4. Continue until all reachable nodes are visited **Time Complexity:** $O(V + E)$

where V = vertices, E = edges

Use Cases:

- Detecting cycles in graphs
- Topological sorting
- Solving maze problems
- Game AI (exploring game trees)

5. System Design

5.1 Program Structure

GraphVisualizer Class

```
|— __init__()      # Initialize graph
|— add_edge()      # Add connections
|— create_sample_graph() # Load default graph
|— display_graph_structure() # Show in terminal
|— draw_graph()    # Visualize with colors
|— bfs()           # BFS implementation
|— dfs()           # DFS implementation
```

5.2 Color Scheme

The visualization uses three colors to represent node states:

- **Red:** Current node being explored
- **Green:** Already visited nodes
- **Blue:** Not yet visited nodes

This color coding helps users track the algorithm's progress visually.

5.3 User Interaction Flow

Start Program

↓

Choose Graph Type

|—— Sample Graph → Load default graph

|—— Custom Graph → User enters edges



Display Graph Structure



Select Algorithm (BFS/DFS/Both)



Enter Starting Node



Watch Visualization



View Results



Repeat or Exit

6. Features

6.1 Core Features

1. **Sample Graph** ○ Pre-built 8-node graph for quick demonstration ○ Structured as a binary tree with connections
2. **Custom Graph Creation** ○ Users can create their own graphs ○ Simple edge input format (node1 node2) ○ Flexible graph structures
3. **Dual Algorithm Support** ○ BFS visualization ○ DFS visualization ○ Option to run both sequentially for comparison
4. **Terminal Display** ○ ASCII art representation of graph structure ○ List of all node connections ○ Clear visual layout

- 5. **Step-by-Step Visualization** ○ Animated
 - graph traversal ○ Pause between steps for observation ○ Console output showing traversal order

7. Sample Graph Structure

The default graph consists of 8 nodes (0-7) with the following structure:

```
0
/\
1 2
/\ / \
3 4 5 6
\ /
7
```

Connections:

- Node 0: Connected to 1, 2
- Node 1: Connected to 0, 3, 4
- Node 2: Connected to 0, 5, 6
- Node 3: Connected to 1, 7
- Node 4: Connected to 1, 7
- Node 5: Connected to 2
- Node 6: Connected to 2
- Node 7: Connected to 3, 4

This structure demonstrates various graph patterns including branching and convergence.

8. Implementation Details

8.1 Graph Representation

The graph is stored as an adjacency list using a Python dictionary:

```
graph = {
    0: [1, 2],
```

```
1: [0, 3, 4],
2: [0, 5, 6],
# ... and so on
}
```

8.2 Visualization Timing

Each step has a 0.8-second pause (`plt.pause(0.8)`) to allow users to observe the algorithm's progress without it being too fast or too slow.

8.3 Graph Layout

NetworkX's `spring_layout` algorithm is used to position nodes aesthetically. The layout is calculated once and reused to maintain consistency throughout the visualization.

9. Results and Output

9.1 BFS Example Output

Starting from node 0:

Traversal Order: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$

BFS explores level by level, visiting all nodes at distance 1 before moving to distance 2.

9.2 DFS Example Output

Starting from node 0:

Traversal Order: $0 \rightarrow 1 \rightarrow 3 \rightarrow 7 \rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow 6$

DFS goes deep into one branch before exploring others, showing a different traversal pattern.

9.3 Visual Output

The matplotlib window shows:

- Nodes as circles with numbers
- Edges as gray lines
- Dynamic color changes during traversal
- Title showing current algorithm and status